

Jacobian–Vector and Vector–Jacobian Products

Reading Materials for COMP532 (UoL)

February 22, 2026

Contents

1	Introduction	2
2	Jacobian as a Linear Map	2
3	Jacobian–Vector Product (JVP)	2
3.1	Definition	2
3.2	Directional Derivative Interpretation	3
4	Vector–Jacobian Product (VJP)	3
4.1	Definition	3
4.2	Scalarization Identity	3
5	Automatic Differentiation Model and Complexity Analysis	4
5.1	A computational graph model	4
5.2	Forward-mode AD and JVP complexity	5
5.3	Reverse-mode AD and VJP complexity	5
5.4	Cost of constructing the full Jacobian: where the m or n factors appear	6
5.5	Summary table	7
5.6	Interpretation and scaling regimes	7
6	Example: Polynomial Map	7
6.1	JVP	7
6.2	VJP	7
7	Fundamental Insight	7
8	Conclusion	8

1 Introduction

Let

$$f : \mathbb{R}^n \rightarrow \mathbb{R}^m$$

be a continuously differentiable function. Its Jacobian at $x \in \mathbb{R}^n$ is the linear map

$$J_f(x) : \mathbb{R}^n \rightarrow \mathbb{R}^m,$$

represented in coordinates by the matrix

$$J_f(x) = \left[\frac{\partial f_i}{\partial x_j}(x) \right]_{\substack{j=1,\dots,n \\ i=1,\dots,m}} \in \mathbb{R}^{m \times n}.$$

In large-scale applications (e.g., deep learning), the explicit construction of $J_f(x)$ is typically computationally infeasible. Instead, one computes its *action* on vectors:

- Jacobian–Vector Product (JVP): $J_f(x)v$
- Vector–Jacobian Product (VJP): $v^\top J_f(x)$

This report presents a formal derivation of both operations, their computational structure under automatic differentiation, and a complexity analysis demonstrating why they are significantly cheaper than constructing the full Jacobian.

2 Jacobian as a Linear Map

The derivative of f at x is the linear map $Df(x)$ satisfying

$$f(x+h) = f(x) + Df(x)[h] + o(\|h\|).$$

Identifying $Df(x)$ with its matrix representation $J_f(x)$ yields:

$$Df(x)[h] = J_f(x)h.$$

Thus, the Jacobian is not merely a matrix of partial derivatives, but the matrix representation of a linear transformation between tangent spaces.

3 Jacobian–Vector Product (JVP)

3.1 Definition

For a tangent vector $v \in \mathbb{R}^n$, the Jacobian–vector product is:

$$J_f(x)v.$$

Componentwise:

$$(J_f(x)v)_i = \sum_{j=1}^n \frac{\partial f_i}{\partial x_j}(x)v_j.$$

3.2 Directional Derivative Interpretation

Define the curve

$$\gamma(\epsilon) = x + \epsilon v.$$

Then:

$$J_f(x)v = \left. \frac{d}{d\epsilon} f(\gamma(\epsilon)) \right|_{\epsilon=0}.$$

Proof. By definition of the derivative,

$$f(x + \epsilon v) = f(x) + Df(x)[\epsilon v] + o(\epsilon).$$

Since $Df(x)$ is linear,

$$Df(x)[\epsilon v] = \epsilon Df(x)[v].$$

Dividing by ϵ and taking the limit gives

$$\left. \frac{d}{d\epsilon} f(x + \epsilon v) \right|_{\epsilon=0} = Df(x)[v] = J_f(x)v.$$

□

Thus, the JVP is the directional derivative of f at x along v .

4 Vector–Jacobian Product (VJP)

4.1 Definition

For a cotangent vector $v \in \mathbb{R}^m$, define the vector–Jacobian product as the row vector:

$$v^\top J_f(x).$$

Componentwise:

$$(v^\top J_f(x))_j = \sum_{i=1}^m v_i \frac{\partial f_i}{\partial x_j}(x).$$

4.2 Scalarization Identity

Define the scalar function:

$$g(x) = v^\top f(x).$$

Treating the gradient $\nabla g(x)$ as a standard column vector in $\mathbb{R}^n$, we have:

$$\nabla g(x) = J_f(x)^\top v.$$

Equivalently, the VJP (a row vector) is the transpose of this gradient:

$$v^\top J_f(x) = \nabla g(x)^\top.$$

Proof. By definition,

$$g(x) = \sum_{i=1}^m v_i f_i(x).$$

Taking the partial derivative with respect to x_j :

$$\frac{\partial g}{\partial x_j} = \sum_{i=1}^m v_i \frac{\partial f_i}{\partial x_j}(x).$$

Arranging these partial derivatives into a column vector yields $\nabla g(x) = J_f(x)^\top v$. Taking the transpose of both sides yields $v^\top J_f(x) = \nabla g(x)^\top$. $\square$

Hence, the VJP computes the gradient of a scalar-valued function formed by a linear combination of the outputs.

5 Automatic Differentiation Model and Complexity Analysis

5.1 A computational graph model

Assume $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is implemented as a straight-line program (computational graph) with L primitive operations. Concretely, define intermediate states

$$z^{(0)} := x \in \mathbb{R}^n, \quad z^{(k)} := \phi_k(z^{(p(k))}), \quad k = 1, \dots, L,$$

where each ϕ_k is a primitive (e.g., addition, multiplication, sin, matrix multiply, convolution) and $p(k)$ indexes the operands used at node k (for simplicity we suppress multi-argument notation; the analysis extends verbatim).

Let $T_{\text{fwd}}(f; x)$ denote the runtime of one *forward* evaluation producing $f(x)$.

Saved intermediates (a.k.a. “tape” in AD literature). Reverse-mode AD requires certain intermediate values from the forward pass to compute local gradients in the backward pass. We denote by

$$S_{\text{save}}(f; x)$$

the memory needed to store these *saved intermediates* (activations, inputs to primitives, and graph connectivity).

PyTorch note. In PyTorch, this concept corresponds to the *autograd graph* created during the forward pass and the *saved tensors* stored for the backward pass. In custom `torch.autograd.Functions`, these are the tensors saved via `ctx.save_for_backward` and later accessed through `ctx.saved_tensors`. The AD literature often calls this stored forward information a “tape”, but PyTorch users more commonly say “saved tensors” or “autograd graph”.

Abstract cost parameter. We summarize forward cost by a single quantity

$$C_f := T_{\text{fwd}}(f; x).$$

In many settings $C_f = \Theta(L)$, but importantly C_f typically depends on problem dimensions (n, m) through L and through the cost of certain primitives (e.g., matrix multiplication).

Examples of how n, m enter C_f .

- **Dense linear map** $f(x) = Wx$ with $W \in \mathbb{R}^{m \times n}$:

$$C_f = \Theta(mn).$$

- **Elementwise map** $f(x) = \sigma(x)$ with $m = n$ and σ applied coordinatewise:

$$C_f = \Theta(n).$$

- **Typical neural network layer** (dense or convolution) often has C_f proportional to the number of multiply-adds, which scales with widths/channels that determine n, m .

Thus, C_f is not strictly “independent” of n, m ; rather, it captures the implementation-dependent arithmetic complexity of evaluating f .

5.2 Forward-mode AD and JVP complexity

Forward-mode automatic differentiation computes the *push forward* of a tangent direction $v \in \mathbb{R}^n$:

$$\text{JVP}(f; x, v) := J_f(x) v.$$

Operationally, forward mode augments each primal variable z with a tangent $\dot{z}$ and applies local chain rules. For each primitive ϕ_k , one computes simultaneously:

$$z^{(k)} = \phi_k(\cdot), \quad \dot{z}^{(k)} = D\phi_k(\cdot) [\text{inputs}].$$

[Cost of a single JVP] Assuming each primitive ϕ_k can be differentiated at a constant-factor overhead relative to its primal evaluation, the runtime of computing $(f(x), J_f(x)v)$ by forward-mode AD satisfies

$$T_{\text{jvp}}(f; x, v) = \Theta(C_f).$$

Proof sketch. Forward mode evaluates the same computational graph once. At each node, in addition to the primal computation, it computes one tangent value via the local derivative of ϕ_k . Under the stated assumption (standard in AD cost models), the derivative propagation costs at most a constant multiple of the primal operation cost. Summing over all nodes gives a constant-factor increase over the forward runtime, hence $\Theta(C_f)$. $\square$

Where n appears. The direction $v \in \mathbb{R}^n$ is part of the input to JVP, but it does *not* cause an additional factor of n in the runtime: one computes the directional derivative along *one* direction v . To obtain the *full* Jacobian via forward mode, one would need n independent directions (basis vectors).

5.3 Reverse-mode AD and VJP complexity

Reverse-mode automatic differentiation computes the *pullback* of a cotangent $u \in \mathbb{R}^m$:

$$\text{VJP}(f; x, u) := u^\top J_f(x) \in \mathbb{R}^{1 \times n}.$$

Equivalently, define the scalarization $g(x) := u^\top f(x)$, then $\nabla g(x)^\top = u^\top J_f(x)$.

Reverse mode runs:

1. A forward pass computing and storing required intermediates (the tape).
2. A backward pass propagating adjoints $\bar{z} := \partial g / \partial z$ from outputs to inputs.

[Cost of a single VJP] Assuming each primitive ϕ_k admits a local reverse-mode rule whose cost is at most a constant multiple of the primal cost, the runtime of computing $u^\top J_f(x)$ by reverse-mode AD satisfies

$$T_{\text{vjp}}(f; x, u) = \Theta(C_f),$$

and the additional memory for reverse-mode is

$$S_{\text{vjp}}(f; x) = \Theta(S_{\text{save}}(f; x)),$$

where $S_{\text{save}}(f; x)$ denotes the memory for saved intermediates.

Proof sketch. The forward pass costs C_f by definition. The backward sweep visits each node once and applies a local adjoint rule. Under the constant-factor assumption, the total backward cost is $O(C_f)$. A lower bound $\Omega(C_f)$ holds because one must at least traverse the computation to produce $f(x)$. Thus total runtime is $\Theta(C_f)$. The backward pass requires intermediate values from the forward pass; the memory needed to store them is $S_{\text{save}}(f; x)$, yielding the stated bound. $\square$

Where m appears. The cotangent $u \in \mathbb{R}^m$ is provided *once*. Computing a single VJP does *not* incur a factor of m . To obtain the full Jacobian via reverse mode, one would need m independent cotangents (basis vectors).

5.4 Cost of constructing the full Jacobian: where the m or n factors appear

The full Jacobian is the matrix whose i -th row is $\nabla f_i(x)^\top$ and whose j -th column is the directional derivative along e_j .

Via reverse mode (row-by-row). To compute all rows, run reverse-mode m times with cotangents $u = e_i \in \mathbb{R}^m$:

$$J_f(x) = \begin{bmatrix} e_1^\top J_f(x) \\ \vdots \\ e_m^\top J_f(x) \end{bmatrix}.$$

By Theorem 5.3,

$$T_{\text{jac,rev}}(f; x) = \Theta(m C_f), \quad S_{\text{jac,rev}}(f; x) \approx \Theta(S_{\text{save}}(f; x)),$$

where the saved intermediates can often be reused per row if x is fixed, but runtime still scales linearly in m .

Via forward mode (column-by-column). To compute all columns, run forward-mode n times with tangents $v = e_j \in \mathbb{R}^n$:

$$J_f(x) = [J_f(x)e_1 \ \cdots \ J_f(x)e_n].$$

By Theorem 5.2,

$$T_{\text{jac,fwd}}(f; x) = \Theta(n C_f).$$

Storage lower bound. Any explicit representation of $J_f(x) \in \mathbb{R}^{m \times n}$ requires $\Theta(mn)$ space.

5.5 Summary table

Let C_f denote the forward evaluation cost, and assume the standard constant-factor overhead model for local derivative rules. Then:

Quantity	Time Complexity	Space Complexity
Forward evaluation $f(x)$	$\Theta(C_f)$	(implementation-dependent)
Single JVP $J_f(x)v$	$\Theta(C_f)$	(small overhead)
Single VJP $u^\top J_f(x)$	$\Theta(C_f)$	$\Theta(S_{\text{save}}(f; x))$
Full Jacobian (reverse)	$\Theta(mC_f)$	$\Theta(mn)$ to store
Full Jacobian (forward)	$\Theta(nC_f)$	$\Theta(mn)$ to store

5.6 Interpretation and scaling regimes

- A *single* JVP or VJP evaluates the action of the Jacobian as a linear operator and costs on the order of one pass through the computational graph.
- The factors m and n appear when one requests *all* rows or *all* columns of $J_f(x)$, i.e., when constructing the full Jacobian, which requires m VJPs or n JVPs.
- Which full-Jacobian strategy is cheaper depends on the regime: reverse is favorable when $m \ll n$; forward is favorable when $n \ll m$.

6 Example: Polynomial Map

Let

$$f(x_1, x_2) = \begin{bmatrix} x_1 x_2 \\ x_1 + \sin(x_2) \end{bmatrix}.$$

Jacobian:

$$J_f(x) = \begin{bmatrix} x_2 & x_1 \\ 1 & \cos(x_2) \end{bmatrix}.$$

6.1 JVP

$$J_f(x) \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} x_2 v_1 + x_1 v_2 \\ v_1 + \cos(x_2) v_2 \end{bmatrix}.$$

6.2 VJP

$$\begin{bmatrix} v_1 & v_2 \end{bmatrix} J_f(x) = \begin{bmatrix} v_1 x_2 + v_2 \\ v_1 x_1 + v_2 \cos(x_2) \end{bmatrix}.$$

7 Fundamental Insight

The Jacobian is a linear operator. Automatic differentiation computes:

$$Df(x)[v] \quad \text{or} \quad Df(x)^*[v],$$

without materializing $Df(x)$ itself.

This reduces complexity from $\Theta(mn)$ storage and $\Theta(\min(m, n)C_f)$ computation down to $\Theta(C_f)$ per directional/adjoint query.

8 Conclusion

- The JVP computes directional derivatives via forward mode.
- The VJP computes gradients of scalar combinations via reverse mode.
- Both operations require time proportional to one evaluation of f .
- Neither requires explicit Jacobian construction.
- This property is fundamental to enabling large-scale deep learning.

References

- [1] A. Griewank and A. Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. Society for Industrial and Applied Mathematics (SIAM), 2nd edition, 2008. (*The definitive theoretical text on AD complexity and algorithms.*)
- [2] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind. Automatic Differentiation in Machine Learning: A Survey. *Journal of Machine Learning Research*, 18(153):1–43, 2018.
- [3] PyTorch Documentation. *Automatic Differentiation with torch.autograd*. https://pytorch.org/tutorials/beginner/basics/autogradqs_tutorial.html
- [4] PyTorch Documentation. *Autograd Mechanics*. <https://pytorch.org/docs/stable/notes/autograd.html>
- [5] A. Paszke, S. Gross, S. Chintala, G. Chanan, et al. Automatic differentiation in PyTorch. *NIPS Autodiff Workshop*, 2017.